

DATABASE MANAGEMENT SYSTEMS

Quick Reference Notes • All 11 Topics

01 ER Model

[Design]

Entity-Relationship Model	Represents real-world data as Entities (objects), Attributes (properties), and Relationships (associations) before building a database.
ER Diagrams	Rectangle = Entity Ellipse = Attribute Diamond = Relationship Double ellipse = Multivalued Dashed ellipse = Derived
Constraints & Keys	Cardinality: 1:1, 1:N, M:N. Participation: Total (every entity must participate) vs Partial. Keys uniquely identify entities. ER maps to tables: entity sets become tables, relationships become foreign keys.

02 Relational Model

[Foundation]

Relational Algebra	Procedural query language: σ (select rows), π (project columns), $\bowtie$ (natural join), $\cup$ $\cap$ $-$ (set operations), $\times$ (cartesian product), ρ (rename).
Relational Calculus	Non-procedural. Tuple Calculus: $\{t \mid \text{condition on } t\}$. Domain Calculus: uses domain variables. Equivalent in power to relational algebra (Codd's theorem).
Integrity Constraints & Keys	Entity integrity: PK cannot be NULL. Referential integrity: FK must match a PK or be NULL. Candidate key = minimal superkey. Primary key = chosen candidate. Foreign key = links two tables.

03 SQL

[Language]

DDL / DML / DCL	DDL: CREATE, ALTER, DROP (structure). DML: SELECT, INSERT, UPDATE, DELETE (data). DCL: GRANT, REVOKE (permissions). TCL: COMMIT, ROLLBACK (transactions).
Queries, Joins & Nested Queries	Joins: INNER, LEFT, RIGHT, FULL OUTER, SELF, CROSS JOIN. Subqueries can appear in WHERE / FROM / SELECT. Correlated subqueries re-execute for each outer row.
Aggregate Functions	COUNT, SUM, AVG, MIN, MAX. GROUP BY creates groups; HAVING filters groups (like WHERE but post-aggregation). NULL values are ignored in aggregation.
Views & Triggers	View: virtual table stored as a query (CREATE VIEW). Trigger: auto-executes on INSERT / UPDATE / DELETE events — BEFORE or AFTER. Used to enforce business rules.

04 Functional Dependencies

[Theory]

Definition & Concepts	$A \rightarrow B$ means A functionally determines B: same A value always gives the same B value. E.g., StudentID $\rightarrow$ Name. Trivial FD: $B \subseteq A$.
----------------------------------	--

Closure of Attributes	$F^+ =$ all FDs derivable from set F . Attribute closure $(A)^+ =$ all attributes determined by A using given FDs. If $(A)^+ =$ all attributes, then A is a superkey.
Armstrong's Axioms	Reflexivity: $B \subseteq A \Rightarrow A \rightarrow B$. Augmentation: $A \rightarrow B \Rightarrow AC \rightarrow BC$. Transitivity: $A \rightarrow B$ and $B \rightarrow C \Rightarrow A \rightarrow C$. These are sound and complete.

05 Normalization		[Design]
1NF, 2NF, 3NF	1NF: atomic values, no repeating groups. 2NF: 1NF + no partial dependency (non-key attr depends on full PK, not part of it). 3NF: 2NF + no transitive dependency (non-key determines non-key).	
BCNF	Boyce-Codd NF: stricter than 3NF. For every FD $X \rightarrow Y$, X must be a superkey. Eliminates all FD-based anomalies but may not preserve all dependencies.	
Lossless Join & Dependency Preservation	Lossless: decomposed tables rejoin to give original (no spurious tuples). Dependency preservation: all original FDs enforceable in decomposed schema. BCNF may sacrifice the latter.	

06 Transactions		[Concurrency]
ACID Properties	Atomicity: all or nothing. Consistency: valid state before and after. Isolation: concurrent transactions appear serial. Durability: committed changes survive crashes.	
Schedules	Serial: transactions execute one after another — always correct but slow. Non-serial (concurrent): interleaved — fast but may cause dirty read, lost update, or unrepeatable read anomalies.	
Conflict & View Serializability	Conflict serializable: equivalent to serial by swapping non-conflicting ops. Test: precedence graph with no cycles. View serializable: broader (allows blind writes) but NP-hard to test.	

07 Concurrency Control		[Concurrency]
Lock-based Protocols & 2PL	Shared lock (read), Exclusive lock (write). Two-Phase Locking: growing phase (acquire locks only), shrinking phase (release only). Strict 2PL holds all locks until commit — guarantees conflict serializability.	
Deadlocks	Detection: wait-for graph — cycle = deadlock; abort one transaction to break cycle. Prevention: Wait-Die (older waits or dies) or Wound-Wait (older wounds younger). Timeout is simplest approach.	
Timestamp Ordering	Each transaction gets a timestamp at start. Read/write operations are validated against per-object timestamps. Older timestamp = higher priority. No locks needed; rollback possible (starvation risk).	

08 Database Recovery		[Reliability]
Log-based Recovery	Write-Ahead Logging (WAL): log record written before the actual DB update. On crash: Undo incomplete transactions, Redo committed ones. Log records: , , , .	

Checkpoints	Periodic sync of log and dirty buffer pages to disk. On crash, recovery starts from last checkpoint — drastically reduces redo work. Fuzzy checkpoints allow transactions to run during the checkpoint.
Shadow Paging	Maintains two page tables: current (active) and shadow (last committed). Updates go to current; shadow is untouched. On commit, current becomes shadow. Simple but expensive for large databases.

09 Indexing & Hashing

[
Performance
]

B Trees & B+ Trees	B Tree: keys and data pointers in all nodes. B+ Tree: only leaves hold data records, linked as a list — faster range queries. Internal nodes are pure index. Most RDBMS use B+ trees.
Static & Dynamic Hashing	Static: fixed buckets; overflow handled by chaining. Dynamic: Extendible hashing (directory doubles on overflow) or Linear hashing (splits linearly). Dynamic adapts to growing data without full rehash.

10 File Organization

[Storage]

Heap Files	Records stored in insertion order — no sorting. Fast insert $O(1)$, slow search $O(n)$. Good for bulk loads or when all records need to be scanned frequently.
Sequential & Index Sequential	Sequential: records physically sorted by key — great for range scans, slow inserts (shifting needed). ISAM (Index Sequential): sparse index over sorted file. Overflow chains handle inserts without resorting.

11 Query Processing & Optimization

[
Performance
]

Query Evaluation	Pipeline: SQL parser → relational algebra tree → query optimizer → execution plan → result. Operators (scan, join, sort, aggregate) can be pipelined (stream results) or materialized (write to disk).
Query Optimization Techniques	Heuristic: push selections and projections down early to reduce tuple count ASAP. Cost-based: estimate I/O + CPU cost for alternative plans using statistics (cardinality, histograms). Pick the cheapest plan.